

Leadflow Mobile API

Integration guide for the Kotlin Android application.

1. Base URL

Local development:

```
http://10.0.2.2:8000/api
```

10.0.2.2 points to the host machine from the Android emulator. For a physical device, replace it with the computer's LAN IP, for example `http://192.168.1.20:8000/api`.

Production example:

```
https://crm.example.com/api
```

All requests and responses use JSON except bulk upload, which uses multipart form data.

2. Authentication

Login

```
POST /login
Content-Type: application/json
Accept: application/json
```

Request:

```
{
  "email": "admin@example.com",
  "password": "password"
}
```

Success 200:

```
{
  "token": "1|long-sanctum-token",
  "user": {
    "id": 1,
    "name": "Aarav Mehta",
    "email": "admin@example.com"
  }
}
```

```
}  
}
```

Store the token securely. Send it on every protected request:

```
Authorization: Bearer 1|long-sanctum-token  
Accept: application/json
```

Logout

```
POST /logout  
Authorization: Bearer {token}
```

Response 200 :

```
{ "message": "Logged out" }
```

3. Common errors

Validation errors return 422 :

```
{  
  "message": "The mobile number field is required.",  
  "errors": {  
    "mobile_number": ["The mobile number field is required."]  
  }  
}
```

Other common responses:

Status	Meaning
401	Missing or invalid Sanctum token
404	Resource does not exist
422	Invalid data, duplicate mobile number, or invalid lookup name
413	Upload is larger than 10 MB

4. Lead fields

Field	Type	Required	Values / notes
id	integer	response	Lead ID

Field	Type	Required	Values / notes
full_name	string	yes	Maximum 255 characters
mobile_number	string	yes	Unique across leads
email	string	no	Valid email
company_name	string	no	
address	string	no	
city	string	no	
state	string	no	
pincode	string	no	Maximum 12 characters
lead_source_id	integer	no	ID from /lead-sources
service_id	integer	no	ID from /services
status	string	no	New , Contacted , Interested , Follow-up , Converted , Lost
priority	string	no	High , Medium , Low
next_followup_at	ISO date	no	Example: 2026-09-20T14:30:00+05:30
remarks	string	no	

5. Dashboard

```
GET /dashboard
Authorization: Bearer {token}
```

Response 200:

```
{
  "total_leads": 42,
  "today_followups": 5,
  "overdue_followups": 3,
  "converted": 8,
  "lost": 2,
  "sources": [
    {
      "lead_source_id": 1,
      "total": 18,
      "source": { "id": 1, "name": "Website", "color": "#4DA3A7" }
    }
  ]
}
```

```
]
}
```

6. Leads

List, search, and filter

```
GET /leads?page=1&per_page=15&search=acme&status=Interested&lead_source_id=2
```

Supported query parameters:

- `page` : Page number, default `1` .
- `per_page` : Results per page, default `15` .
- `search` : Searches `full_name` , `mobile_number` , and `company_name` .
- `status` : One of the supported statuses.
- `lead_source_id` : Filter by source ID.

Response `200` is Laravel pagination:

```
{
  "current_page": 1,
  "data": [
    {
      "id": 1,
      "full_name": "Neha Sharma",
      "mobile_number": "9876543210",
      "status": "Interested",
      "priority": "High",
      "lead_source_id": 1,
      "service_id": 2,
      "source": { "id": 1, "name": "Website", "color": "#4DA3A7" },
      "service": { "id": 2, "name": "Mobile App" }
    }
  ],
  "last_page": 1,
  "per_page": 15,
  "total": 1
}
```

Lead details

```
GET /leads/{leadId}
```

Returns the lead, source, service, and follow-up history. Follow-ups are newest first.

Add lead

```
POST /leads
Content-Type: application/json
```

```
{
  "full_name": "Neha Sharma",
  "mobile_number": "9876543210",
  "email": "neha@example.com",
  "company_name": "Acme Retail",
  "city": "Mumbai",
  "state": "Maharashtra",
  "pincode": "400001",
  "lead_source_id": 1,
  "service_id": 2,
  "status": "New",
  "priority": "High",
  "next_followup_at": "2026-09-20T14:30:00+05:30",
  "remarks": "Asked for an app development estimate."
}
```

Success: 201 with the created lead.

Edit lead

```
PUT /leads/{leadId}
Content-Type: application/json
```

Send the same lead payload. `full_name` and `mobile_number` remain required; the current lead's own mobile number is allowed.

Success: 200 with the updated lead.

Delete lead

```
DELETE /leads/{leadId}
```

Response 200:

```
{ "message": "Lead deleted" }
```

7. Follow-ups

Add follow-up

```
POST /leads/{leadId}/followups
Content-Type: application/json
```

```
{
  "followup_at": "2026-09-18T11:00:00+05:30",
  "note": "Discussed the proposal. Client requested a revised timeline.",
  "type": "Call",
  "next_followup_at": "2026-09-21T15:00:00+05:30",
  "status_after": "Follow-up"
}
```

`type` values: `Call`, `WhatsApp`, `Meeting`, `Email`.

`status_after` may be: `New`, `Contacted`, `Interested`, `Follow-up`, `Converted`, or `Lost`.

Success: `201` with the follow-up. The lead's status and next follow-up are updated automatically.

Today's follow-ups

```
GET /followups/today
```

Returns an array of leads whose `next_followup_at` is today, ordered by time.

8. Bulk upload

```
POST /leads/import
Content-Type: multipart/form-data
```

Multipart field:

```
file = leads.xlsx
```

Accepted files: `.csv`, `.txt`, `.xlsx`, `.xls`. Maximum size: 10 MB.

Recommended CSV headings:

```
full_name,mobile_number,email,company_name,city,state,pincode,lead_source,interested_service,remarks
```

Duplicate `mobile_number` rows are skipped. Response `200`:

```
{ "message": "Import complete" }
```

9. Lead sources

All endpoints require authentication.

```
GET    /lead-sources
POST   /lead-sources
GET    /lead-sources/{id}
PUT    /lead-sources/{id}
DELETE /lead-sources/{id}
```

Create/update request:

```
{
  "name": "Google Ads",
  "color": "#4DA3A7"
}
```

The list and detail responses include `leads_count` .

10. Interested services

All endpoints require authentication.

```
GET    /services
POST   /services
GET    /services/{id}
PUT    /services/{id}
DELETE /services/{id}
```

Create/update request:

```
{
  "name": "CRM Software"
}
```

The list and detail responses include `leads_count` .

Deleting a source or service is allowed. Because the database foreign keys use `nullOnDelete` , existing leads remain and their `lead_source_id` or `service_id` becomes `null` .

11. Kotlin setup

Recommended dependencies:

```
dependencies {
    implementation("com.squareup.retrofit2:retrofit:2.11.0")
    implementation("com.squareup.retrofit2:converter-gson:2.11.0")
    implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")
    implementation("androidx.datastore:datastore-preferences:1.1.1")
    implementation("androidx.room:room-runtime:2.6.1")
}
```

```
kapt("androidx.room:room-compiler:2.6.1")
}
```

Kotlin models

```
data class LoginRequest(val email: String, val password: String)
data class LoginResponse(val token: String, val user: User)
data class User(val id: Long, val name: String, val email: String)

data class Lead(
    val id: Long,
    val full_name: String,
    val mobile_number: String,
    val email: String?,
    val company_name: String?,
    val address: String?,
    val city: String?,
    val state: String?,
    val pincode: String?,
    val lead_source_id: Long?,
    val service_id: Long?,
    val status: String,
    val priority: String,
    val next_followup_at: String?,
    val remarks: String?,
    val source: LeadSource?,
    val service: Service?,
    val followups: List<Followup> = emptyList()
)

data class LeadSource(val id: Long, val name: String, val color: String?, val leads_count: Int? = null)
data class Service(val id: Long, val name: String, val leads_count: Int? = null)
data class Followup(
    val id: Long,
    val lead_id: Long,
    val followup_at: String,
    val note: String,
    val type: String,
    val next_followup_at: String?,
    val status_after: String?
)

data class FollowupRequest(
    val followup_at: String,
    val note: String,
    val type: String,
    val next_followup_at: String?,
    val status_after: String?
)

data class NameRequest(val name: String)
data class NameColorRequest(val name: String, val color: String? = null)
data class MessageResponse(val message: String)
data class LeadRequest(
    val full_name: String,
    val mobile_number: String,
    val email: String? = null,
```

```

    val company_name: String? = null,
    val address: String? = null,
    val city: String? = null,
    val state: String? = null,
    val pincode: String? = null,
    val lead_source_id: Long? = null,
    val service_id: Long? = null,
    val status: String? = null,
    val priority: String? = null,
    val next_followup_at: String? = null,
    val remarks: String? = null
)
data class LeadPage(
    val current_page: Int,
    val data: List<Lead>,
    val last_page: Int,
    val per_page: Int,
    val total: Int
)
data class SourceBreakdown(
    val lead_source_id: Long?,
    val total: Int,
    val source: LeadSource?
)
data class DashboardSummary(
    val total_leads: Int,
    val today_followups: Int,
    val overdue_followups: Int,
    val converted: Int,
    val lost: Int,
    val sources: List<SourceBreakdown>
)

```

Use `@SerializedName` if Kotlin property names are converted to camelCase by your project.

Retrofit API

```

interface LeadflowApi {
    @POST("login")
    suspend fun login(@Body request: LoginRequest): LoginResponse

    @POST("logout")
    suspend fun logout(): MessageResponse

    @GET("dashboard")
    suspend fun dashboard(): DashboardSummary

    @GET("leads")
    suspend fun leads(
        @Query("page") page: Int = 1,
        @Query("per_page") perPage: Int = 15,
        @Query("search") search: String? = null,
        @Query("status") status: String? = null,
        @Query("lead_source_id") sourceId: Long? = null
    ): LeadPage
}

```

```

@GET("leads/{id}")
suspend fun lead(@Path("id") id: Long): Lead

@POST("leads")
suspend fun createLead(@Body lead: LeadRequest): Lead

@PUT("leads/{id}")
suspend fun updateLead(@Path("id") id: Long, @Body lead: LeadRequest): Lead

@DELETE("leads/{id}")
suspend fun deleteLead(@Path("id") id: Long): MessageResponse

@POST("leads/{id}/followups")
suspend fun addFollowup(@Path("id") id: Long, @Body request: FollowupRequest): Followup

@GET("followups/today")
suspend fun todayFollowups(): List<Lead>

@GET("lead-sources")
suspend fun sources(): List<LeadSource>

@POST("lead-sources")
suspend fun createSource(@Body request: NameColorRequest): LeadSource

@PUT("lead-sources/{id}")
suspend fun updateSource(@Path("id") id: Long, @Body request: NameColorRequest): LeadSource

@DELETE("lead-sources/{id}")
suspend fun deleteSource(@Path("id") id: Long): MessageResponse

@GET("services")
suspend fun services(): List<Service>

@POST("services")
suspend fun createService(@Body request: NameRequest): Service

@PUT("services/{id}")
suspend fun updateService(@Path("id") id: Long, @Body request: NameRequest): Service

@DELETE("services/{id}")
suspend fun deleteService(@Path("id") id: Long): MessageResponse
}

```

Bearer token interceptor

```

class AuthInterceptor(private val tokenStore: TokenStore) : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val token = tokenStore.token()
        val request = chain.request().newBuilder()
            .header("Accept", "application/json")
            .apply { if (!token.isNullOrEmpty()) header("Authorization", "Bearer $token") }
            .build()
        return chain.proceed(request)
    }
}

```

Do not attach the bearer token to the login request if the token store is empty. On a `401`, clear the token and navigate to login.

12. Offline caching and refresh

Recommended Android flow:

1. Store the token in encrypted DataStore or another secure storage.
2. Store leads and follow-ups locally with Room.
3. Display Room data immediately when the screen opens.
4. Trigger `GET /leads` on pull-to-refresh.
5. Replace or upsert local records using the server `id`.
6. Queue offline create/update/follow-up operations in a local outbox table.
7. Retry queued operations when connectivity returns.
8. Treat the server response as authoritative after each successful sync.
9. Use `next_followup_at` and `followup_at` as ISO-8601 timestamps and convert them to the device timezone for display.

13. Local Android testing

Start Laravel on all interfaces:

```
php artisan serve --host=0.0.0.0 --port=8000
```

Android emulator base URL:

```
http://10.0.2.2:8000/api/
```

If using cleartext HTTP in local development, add this temporarily to `AndroidManifest.xml`:

```
<application android:usesCleartextTraffic="true" ... />
```

Use HTTPS in production and remove the cleartext setting.